Technical Audit of the GNN Pipeline (Tab by Tab)

Detailed Documentation, GraphSMOTE, and ImGAGN

February 24, 2026

Contents

1 Scope and methodology

This report documents and audits the GNN pipeline implemented in the repository, focusing on:

- The tab-by-tab workflow of the Streamlit application (`src/graph_builder_app.py`).
- Graph construction, the GNN model, training, and evaluation (`src/graph.py`, `src/gat_model.py`, `src/gnn_main.py`, `src/train_pretrain.py`).
- Class balancing mechanisms: GraphSMOTE and ImGAGN (`src/graphsmote.py`, `src/imgagn.py`).
- Hyperparameter optimization (Optuna) and metric tracking.

The audit includes best-practice checks, information leakage risks, memory/compute considerations, and consistency with configuration options in `src/config.py`.

2 Overall pipeline map

Conceptual summary:

1. **Base data**: gantries, flows, and accidents (`src/utils.py`).
2. **Feature engineering**: snapshot/flow processing (`src/snapshot_pipeline.py`, `src/features.py`).
3. **Feature selection**: manual/UI-guided selection and persistence.
4. **Graph**: `pm` nodes, `temporal/spatial/st_fwd` edges, and edge attributes (`src/graph.py`).
5. **Training**: HeteroGAT + optional TemporalAggregator (`src/gat_model.py`, `src/temporal_head.py`).
6. **Balancing**: GraphSMOTE (embedding-space + z→x + edge generation) and/or ImGAGN (adversarial) (`src/graphsmote.py`, `src/imgagn.py`).
7. **Evaluation**: metrics (F1, AUC, AUPRC, MCC), thresholds, and calibration (`src/gnn_main.py`).
8. **Artifacts**: models, hparams JSON files, augmented graphs, and run history (`Resultados/`, `gnn_history.jsonl`).

3 Pipeline *tab by tab* (Streamlit Graph Builder)

The UI flow is implemented in `src/graph_builder_app.py` and organized in tabs.

3.1 Events

UI function: `_render_events_tab`.
Goal: load and validate accidents, normalize columns, clean data, and prepare the events dataset.

- Load accidents from CSV/selected files and persist them in `st.session_state`.
- Use helpers in `src/utils.py` for column and timestamp normalization.
- Support generation of *Black Spot* reports (later used in *Graph*).

Implication: this tab defines the accident universe used by later labels and temporal splits.

3.2 Feature Engineering

UI function: `_render_feature_engineering`.
Goal: generate or load features per `pm` node.

- Main modes: *Snapshot Features* and *Flow 5 min* (see `src/snapshot_pipeline.py`).
- Feature generation via `compute_pm_features` (`src/features.py`) and/or `SnapshotFeatureBuilder`.
- Output persisted in DuckDB or CSV, synchronized by time window and road segment.

Implication: affects node dimensionality and feature distribution. Time resolution is controlled with `DT_MINUTES` and parameters in `src/config.py`.

3.3 Feature Explorer

UI function: `render_feature_explorer`.
Goal: visual inspection, descriptive statistics, and outlier diagnostics for input variables. **Implication**: quality gate for feature sanity before graph construction.

3.4 Feature Selection

UI function: `_render_feature_selection_tab`.
Goal: select feature subsets for training and persist selected lists/metadata. **Implication**: changes the GNN input space and `pm.x` dimensionality.

3.5 Graph

UI functions: `_render_create_graph`, `_render_load_graph`, `_render_in_memory_graph`.
Goal: build or load the heterogeneous gantry graph.

- Segment selection (manual or black-spot-driven).
- Construction of `pm` (gantry-minute) nodes with normalized features.
- Generation of temporal/spatial edges and edge attributes (see Section 4).
- Creation of `train/val/test_mask` via temporal cutoffs (`_compute_time_cutoffs`).
- Save to `Resultados/highway_graph_*.pt` and memory (`st.session_state`).

3.6 Visual Graph

UI function: `render_visual_graph_tab`.
Goal: visualize topology, degree patterns, and subgraphs for QA.

3.7 Network

UI function: `_render_network_tab`.
Goal: configure model architecture and parameters before training. **Implication**: sets values such as `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`, and checkpointing.

3.8 Optuna

UI function: `_render_optuna_tab`.
Goal: run hyperparameter optimization with Optuna (see Section 9). **Implication**: stores `optuna_hyperparams_*`
and drives auto-selection during training.

3.9 Balance

UI function: `_render_balance_tab`.
Goal: generate balanced graphs with GraphSMOTE or ImGAGN.

- GraphSMOTE: requires a trained model for embeddings and `z→x` decoders.
- ImGAGN: trains adversarial generator and discriminator to create synthetic nodes.

Implication: creates graphs with `is_synthetic`, updates `train_mask`, and can modify effective class balance.

3.10 Training

UI function: `_render_training_tab`.
Goal: train the GNN with optional balanced graph, early stopping, and automatic evaluation.
Backend: `src/gnn_main.py::run_gat_training` and `src/train_pretrain.py::train_minibatch`.

3.11 Evaluation

UI function: `_render_evaluation_tab`.
Goal: load models, evaluate metrics, apply thresholding, and optionally calibrate probabilities.
Backend: `src/gnn_main.py::test` and export utilities.

3.12 History

UI function: `_render_history_tab`.
Goal: experiment registry and traceability (`Resultados/gnn_history.jsonl`).

3.13 Experiments

UI function: `_render_gnn_experiments_tab`.
Goal: run experiment batches, export results, and import ZIP bundles.

4 Graph data model

Nodes: `pm` (gantry-minute).
Targets: `pm.y` (binary label), `pm.is_accident_pm` (direct accident flag).
Masks: `train_mask/val_mask/test_mask/sequence_mask`.

4.1 Edges and attributes

Main edges (`src/graph.py`):

- `temporal`: $P_{i,t} \to P_{i,t+1}$ (same location, next time).
- `spatial`: $P_{i,t} \to P_{i+1,t}$ (next gantry, same time).
- `st_fwd` (optional): $P_{i,t} \to P_{i+1,t+1}$.
- `spatial_back` (optional): reverse of `spatial`.

Edge attributes: feature differences between destination and source (Δx) plus extra channels for `spatial` edges (gradients, shock, distance). For `temporal` and `st_fwd`, extra channels are set to zero to keep dimensions aligned.

4.2 Temporal labeling

Time cutoffs: defined from chronologically sorted accidents (`_compute_time_cutoffs`).
Masks: assigned by minute-level timestamps.
SequenceIndex: temporal sequences with guard band and future horizon (`src/snapshot_sequences.py`).

5 GNN model architecture

Base model: `src/gat_model.py::HeteroGAT`.

- **Layers**: `num_layers` blocks of `HeteroConv` with `GATConvSaveAlpha`.
- **Aggregation**: `aggr1` on first layer, `aggr2` on subsequent layers.
- **Attentions**: captured when `XAI=1`, used for $L2$ regularization.
- **Edge attributes**: validated per batch; missing/misaligned tensors are zero-padded.
- **Checkpointing**: `use_checkpointing` reduces memory usage via recomputation.

5.1 Temporal head (optional)

Class: `src/temporal_head.py::TemporalAggregator`.
Idea: GRU over recent embedding sequences; keeps a cache for nodes not present in the current batch, avoiding gradient leakage.

5.2 GNN variant registry (encoder + temporal + edge processing)

The pipeline now includes a **variant registry** controlled from `src/config.py` (`GNN_VARIANT`, `GNN_TEMPORAL_CACHE_` and instantiated in `src/gnn_main.py` through `_build_gnn_model`, `_build_temporal_head_for_variant`, and `_prime_temporal_cache_if_needed`. In addition, the UI in `src/graph_builder_app.py` exposes a condensed guide in the *Network*, *Training*, *Optuna*, and *Evaluation* tabs. The design separates:

- **Spatial encoder**: base heterogeneous GAT, GAT with edge encoder, or edge-aware alternative.
- **Temporal head**: snapshot-only (no sequence), GRU, temporal Transformer, or attention pooling.
- **Cache protocol**: batch-wise `incremental` caching or `global_epoch` (one global encoder pass per epoch for consistent sequences).

5.2.1 General comparability idea

All variants share the same `SequenceIndex` (built in `src/snapshot_sequences.py` using `sequence_rows`/`target_ro` the same graph, and the same temporal splits. This enables architectural comparisons without changing the target definition or prediction horizon.

"5 minutes before" labeling The labeling logic uses:

$$\text{positive interval} = (t + \gamma,\ t + \gamma + H]$$

where γ is `GUARD_BAND_MINUTES` and H is `HORIZON_MINUTES`. For a strict "5 minutes before" setup, the recommended configuration is:

- `GUARD_BAND_MINUTES = 5`
- `HORIZON_MINUTES = 5` (or 10–20 for a risk window)

in `src/config.py`.

5.2.2 Implemented variants

Implementation: `src/gat_model.py`, `src/temporal_head.py`, `src/temporal_heads.py`, `src/gnn_main.py`.
Selection: via the `GNN_VARIANT` string (normalized in `src/gnn_main.py::_normalize_gnn_variant`).

1. `gat_snapshot` (baseline): `HeteroGAT` without a temporal head. Classification uses the current snapshot embedding.
2. `gat_gru`: `HeteroGAT` + `TemporalGRUHead` (`src/temporal_heads.py`).
3. `gat_transformer`: `HeteroGAT` + `TemporalTransformerHead` (self-attention with learnable positional embedding).
4. `gat_attnpool`: `HeteroGAT` + `TemporalAttnPoolHead` (lightweight, low-cost ablation).
5. `gat_edge_mlp`: `HeteroGATWithEdgeEncoder` (per-edge-type MLP over `edge_attr`) without a temporal head.
6. `gat_edge_mlp_gru` and `gat_edge_mlp_transformer`: combine edge encoding with temporal heads.
7. `gnn_edge_aware`: `HeteroEdgeAware` (built on `TransformerConv`) without a temporal head.
8. `gnn_edge_aware_gru` and `gnn_edge_aware_transformer`: edge-aware variants with temporal modeling.

5.2.3 Edge attribute encoding (`edge_attr`)

The `HeteroGATWithEdgeEncoder` variant adds an edge MLP (`EdgeAttrEncoder`) before graph attention. Motivation:

- Δx channels may have heterogeneous scales;
- a learned mapping can induce a more stable geometry for attention;
- it enables isolated analysis of "better `edge_attr`" vs. "better temporal head".

5.2.4 Temporal heads and sequence assembly

Temporal heads reuse the `TemporalAggregator` infrastructure:

- `update_cache(...)` for global or partial embedding caching.
- `_build_sequence_batch(...)` to reconstruct $[B, L, D]$ tensors using `sequence_rows`.

This allows changing the temporal operator (GRU/Transformer/attention pooling) without changing the training contract in `src/train_pretrain.py` or the model `forward` signature.

5.2.5 Temporal cache strategies (critical implementation point)

- `incremental`: batch-wise cache updates (historical behavior, lower cost, more sampler-dependent).
- `global_epoch`: epoch-level priming using a global encoder pass (`compute_epoch_embeddings`), then train/eval consume a consistent cache.

Recommendation for paper-grade comparisons: prefer `global_epoch` so temporal sequences are aligned to the same embedding reference within each epoch.

5.2.6 Recommended experiment matrix

1. `gat_snapshot` (spatial baseline)
2. `gat_gru`
3. `gat_transformer`
4. `gat_edge_mlp_gru`
5. `gnn_edge_aware_gru` (or `gnn_edge_aware_transformer`)

Keep seed, temporal split, `SequenceIndex`, and evaluation protocol fixed.

5.2.7 Metrics for rare events

For scientific and operational comparison, prioritize:

- **PR-AUC / AUPRC**: primary metric under severe imbalance.
- **ROC-AUC**: secondary reference.
- **Recall at target FAR/FPR**: useful under operational false-alarm constraints.
- **MCC**: robust under strong class asymmetry.

5.2.8 Variant traceability

Training and HPO persist `gnn_variant` and `variant_tag` in hyperparameter sidecars, checkpoints, and structured logs (`Resultados/gat_model_BEST*.pt`, `_hparams.json`, Optuna CSVs). This prevents mixing checkpoints from incompatible architectures.

6 Training and optimization

Entry point: `src/gnn_main.py::run_gat_training`.

6.1 Training flow

1. **Device**: `get_auto_device()` (MPS > CUDA > CPU).
2. **Data load**: `loaded_obj['data']`.
3. **Automatic ImGAGN** (if `AUTO_IMGAGN_PRETRAIN=1`).
4. **GraphSMOTE**: selected by user/flag with detection of existing synthetic nodes.
5. **HPO**: load `optuna_hyperparams_*.csv` or run a new search.
6. **Model**: the `_build_gnn_model` factory selects the encoder/temporal head according to `gnn_variant` and attaches a temporal head when applicable.
7. **z→x decoders**: train if GraphSMOTE is active (`train_z2x_decoders`).
8. **Edge generator**: `RelEdgeGen` for edge reconstruction and synthetic edge creation.

9. **Loss**: `CrossEntropy` or `FocalLoss` (with or without class weights).
10. **Scheduler**: `OneCycleLR`.
11. **Loop**: mini-batch training, validation, early stopping, and best-model checkpointing.

6.2 Mini-batch training and regularization

Function: `src/train_pretrain.py::train_minibatch`.

- Classification on `pm` nodes (first `batch_size` elements from `NeighborLoader`).
- Total loss:
$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_{edge}\,\mathcal{L}_{edge} + \lambda_{att}\,\mathcal{L}_{att}.$$
- $\mathcal{L}_{edge}$: edge reconstruction with negative sampling (when `edge_gen` is active).
- $\mathcal{L}_{att}$: $L2$ attention regularization when `XAI=1`.
- Gradient accumulation (`accumulation_steps=2`) and `grad_clip`.
- AMP support (CUDA only).

7 GraphSMOTE: synthetic node and edge generation

Module: `src/graphsmote.py`.
Principle: synthesize nodes in embedding space and decode them into feature space.

7.1 Step 1: embeddings

- `get_embeddings_minibatch` (offline UI balancing) or `compute_train_embeddings` (training-time, leakage-safe).
- Node-wise ℓ_2 normalization.

7.2 Step 2: SMOTE in embedding space

SMOTE is applied to the minority set:

$$\mathbf{z}_{syn} = (1-\alpha)\,\mathbf{z}_i + \alpha\,\mathbf{z}_j, \quad \alpha \sim U(0,1).$$

- k-NN over minority embeddings (CPU cache or GPU when feasible).
- Key parameters: `GRAPHSMOTE_K`, `smote_k`, `random_state`.

7.3 Step 3: z→x decoders

Class: `Z2XDecoders`.
Training: `train_z2x_decoders` with SmoothL1/MSE, early stopping, and validation.

- One MLP per node type: $\mathbf{x}\_syn = f_{ntype}(\mathbf{z}\_syn)$.
- Respect `train/val_mask` and avoid out-of-split nodes when masks are available.

7.4 Step 4: node insertion

- Concatenate synthetic `x` and `y` into `pm.x` and `pm.y`.
- Update `train/val/test_mask` (new nodes go to train by default).
- Create/update `is_synthetic` and `is_accident_pm`.

7.5 Step 5: synthetic edge generation

Generator: `RelEdgeGen` (relation-specific parameters).
Real-node candidates:

- `GRAPHSMOTE_CONECT=1`: top-K within `train_mask`.
- `GRAPHSMOTE_CONECT=0`: one-hop neighbors of real accident nodes.

Direction:

- `BIDIRECCTION=1`: real $\leftrightarrow$ synthetic.
- `BIDIRECCTION=0`: real $\rightarrow$ synthetic only.

Edge attributes: computed as destination-source Δx (with `delta_feature_idx` when available) and aligned to existing edge-attribute dimensionality.

7.6 Operation modes

- **Offline**: `augment_graph_offline_once` augments once and trains on the augmented graph.
- **Online**: `refresh_synthetics_online` regenerates nodes every `SMOTE_EVERY_N_EPOCHS`.
- **None**: no augmentation.

8 ImGAGN: adversarial generation

Module: `src/imgagn.py`.
Core idea: generate synthetic nodes as convex combinations of minority *training* nodes and train an adversarial discriminator.

8.1 Step 1: homogenization

`to_homogeneous_if_needed` converts `HeteroData` into a homogeneous graph (target `pm`), preserving `edge_attr` and `delta_feature_idx` when present.

8.2 Step 2: define synthetic node count

$$\lambda_1 = \frac{n_{min} + n_g}{n_{maj}}$$
$$\Rightarrow n_g = \max\big(0,\, \lambda_1 n_{maj} - n_{min}\big)$$

The value is capped by `max_new_nodes` to avoid OOM conditions.

8.3 Step 3: generator

Class: `ImGAGNGenerator`.

- Input: noise $\mathbf{z} \sim \mathcal{N}(0, I)$.
- Output: logits over minority training nodes.
- Weights $T = \text{softmax}(G(z))$ and synthetic features $X_g = TX_{min}$.
- Edges: top-K links to real minority nodes according to T.

8.4 Step 4: discriminator

Class: `ImGAGNDiscriminator` (GCN + two heads).

- **Real/fake head** (BCE).
- **Minority/majority head** (BCE).
- Separation term `_mm_separation` to push minority and majority means apart.

8.5 Step 5: adversarial training

For each `epoch`:

1. Sample noise and build synthetic nodes (without gradients).
2. Build augmented graph and train D for `d_steps` mini-batches.
3. Recompute X_g with gradients and optimize G using:

$$\mathcal{L}_G = \mathcal{L}_{rf} + \mathcal{L}_{mi} + \mathcal{L}_{di},$$

   where $\mathcal{L}_{rf}$ pushes toward *real*, $\mathcal{L}_{mi}$ pushes toward *minority*, and $\mathcal{L}_{di}$ pulls embeddings toward minority centroids.

Scalability: uses `NeighborLoader` when available; otherwise falls back to full-batch.

8.6 Step 6: final graph

The augmented graph is rebuilt with the final generator, returning: `x_aug`, `edge_index_aug`, `edge_attr_aug`, and the generated-node block.

9 Optuna (hyperparameter optimization)

Function: `src/gnn_main.py::objective`.

- Objective: maximize validation F0.5 (precision-oriented).
- Search space: `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`.
- Optimizers: Adam/AdamW/RAdam (configurable).

- With GraphSMOTE: search `smote_k`, `target_pos_ratio`, `smote_every_n_epochs`, `lambda_edge`.
- Without GraphSMOTE: `FocalLoss` with `focal_alpha`, `focal_gamma`.

Seed sampling options (train/val):

- *Resample seeds each epoch*: changes subset every epoch (robustness-oriented).
- *Fix subset per trial*: keeps one subset during each trial (lower variance).
- *Load full graph in memory (no sampling)*: uses all `train/val` nodes (equivalent to `sample_train_nodes=0` and `sample_val_nodes=0`); higher memory usage.

10 Evaluation, calibration, and artifacts

- **Evaluation**: `test` computes F1, AUC, AUPRC, MCC, and confusion matrix.
- **Threshold**: `pick_tau_fbeta` searches threshold τ on validation.
- **Calibration**: optional Platt scaling (`AUTOCALIBRATE_PROBS`).
- **Persistence**: models `gat_model_BEST*.pt`, `GraphSMOTE_embeddings_model*.pt`, and sidecar `_hparams.json`.
- **Graph hash**: `_calculate_graph_hash` for traceability.

11 Technical audit and best practices

11.1 Observed strengths

- Temporal split for `train/val/test` plus `sequence_mask` reduces temporal leakage.
- Training-time GraphSMOTE uses **train** embeddings (`compute_train_embeddings`).
- `NeighborLoader` bounds memory and enables large-graph training.
- Cleaning/*coalesce* mechanisms prevent `edge_attr` inconsistencies.
- Hparams and *run_id* persistence improve reproducibility.

11.2 Risks and critical points (updated)

- **Leakage in manual balancing (mitigated)**: `run_graphsmote` now restricts embedding/minority pools to `train_mask`. Keep auditing this condition.
- **Dependence on** `BIDIRECCTION`: synthetic edge direction affects message flow and may bias signal propagation. Kept unchanged for now.
- **Gradient accumulation (tunable)**: `accumulation_steps` is now configurable (config/UI) but still requires hardware-specific tuning.
- **Balancing + class weights**: class weights are disabled when GraphSMOTE/ImGAGN are active (correct), but validation metric monitoring remains required.

11.3 Prioritized recommendations (current status)

1. **Implemented**: in manual balancing (`run_graphsmote`), restrict to `train_mask` to avoid potential leakage.
2. **Implemented**: expose `accumulation_steps` in config/UI and tune by available memory.
3. **Pending/decision**: keep `BIDIRECCTION` unchanged for now; document rationale and evaluate AUPRC/F1 impact if changed later.
4. **Implemented**: in ImGAGN, register `best_train_recall` together with seed and config for traceability.

12 Appendix: file and function references

- Main UI: `src/graph_builder_app.py`.
- Graph construction: `src/graph.py`.
- GNN model: `src/gat_model.py`.
- Temporal head: `src/temporal_head.py`.
- Training/evaluation: `src/gnn_main.py`, `src/train_pretrain.py`.
- GraphSMOTE: `src/graphsmote.py`.
- ImGAGN: `src/imgagn.py`.
- Configuration: `src/config.py`.